

1. Problem Understanding and Formulation

1.1 Problem Definition

Customer churn refers to a situation in which an existing customer stops using a company's services. For a service-based company, losing customers can reduce recurring revenue and increase the cost of acquiring replacement customers. The objective of this project is to develop a machine-learning system that predicts which customers are likely to churn and groups customers with similar characteristics so that retention strategies can be targeted.

1.2 Problem Formulation

The prediction problem is formulated as binary classification. For each customer, the input consists of demographic, service, contract and billing attributes. The target variable is Churn, with Yes representing churn and No representing continued service. A second analytical task is formulated as unsupervised clustering, where customers are grouped without using the churn label.

1.3 Objectives

- Acquire and inspect a suitable public customer-churn dataset.
- Clean missing and inconsistent values and remove irrelevant attributes.
- Perform exploratory data analysis to identify important patterns.
- Train at least three supervised classification algorithms.
- Evaluate models using accuracy, precision, recall, F1-score and confusion matrix.
- Select a suitable final model using quantitative evidence.
- Apply K-means clustering to identify behavioural/service-related customer groups.
- Use PCA to visualize clusters in a lower-dimensional space.
- Identify high-risk groups and recommend suitable retention actions.

1.4 Dataset Description

The selected dataset is the IBM Telco Customer Churn dataset. It contains 7,043 customer records and 21 columns. The variables cover customer demographics, subscribed services, account information, contract details, payment method, tenure and charges. The Churn attribute is the response variable.

Category	Representative attributes
Demographic	Gender, SeniorCitizen, Partner, Dependents
Phone services	PhoneService, MultipleLines
Internet services	InternetService, OnlineSecurity, OnlineBackup, DeviceProtection, TechSupport
Entertainment	StreamingTV, StreamingMovies
Account	tenure, Contract, PaperlessBilling, PaymentMethod
Financial	MonthlyCharges, TotalCharges
Target	Churn

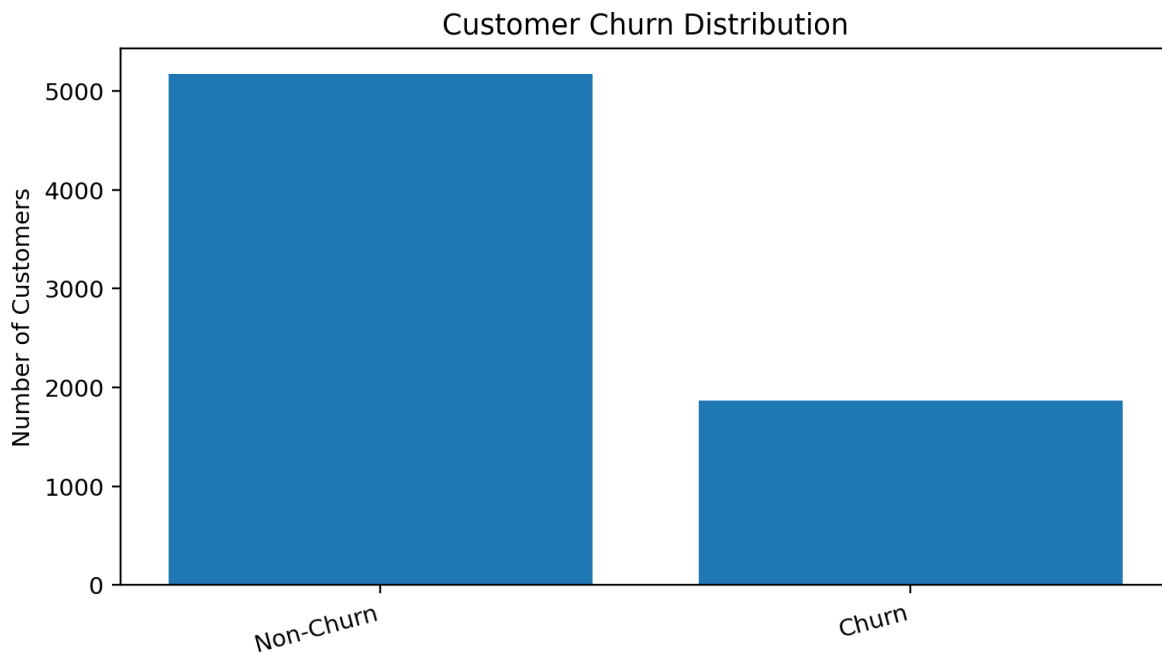


Figure 1. Distribution of churn and non-churn customers.

1.5 Available Information

The dataset provides enough information to examine relationships between customer tenure, service subscriptions, contracts, payment behaviour and churn. The class distribution is imbalanced: 1,869 customers are labelled as churners and 5,174 as non-churners.

1.6 Assumptions and Constraints

- CustomerID is treated as an identifier and removed from modelling.
- TotalCharges is converted from text to numeric form; invalid blank values become missing values.
- Categorical variables are encoded using one-hot encoding.
- Numerical variables are standardized where required.
- An 80:20 stratified train-test split is used.
- The historical dataset is assumed to represent useful churn patterns, although real-world behaviour may change.
- Accuracy is not used alone because the target classes are imbalanced.

2. Application of Course Knowledge

2.1 Supervised Learning

Supervised learning learns a mapping from labelled examples to an output. Here, historical customer records provide both the feature vector X and the known churn label y . The trained model estimates the probability or class of churn for an unseen customer.

2.2 Logistic Regression

Logistic Regression estimates the probability of churn using the sigmoid function:

$$p(y=1|x) = 1 / (1 + e^{-(\beta_0 + \beta^T x)})$$

A probability threshold is used to convert the estimated probability into a class. Logistic Regression is useful because its coefficients provide an interpretable indication of the direction of feature effects.

2.3 Gaussian Naive Bayes

Naive Bayes uses Bayes' theorem: $P(C|X) = P(X|C)P(C)/P(X)$. The classifier assumes that features are conditionally independent given the class. Gaussian Naive Bayes models continuous transformed features with class-specific normal distributions.

2.4 Decision Tree

A Decision Tree creates a sequence of feature-based decisions. One common impurity measure is Gini impurity: $G = 1 - \sum p_i^2$. A split is selected when it improves the purity of the resulting child nodes. Tree depth and minimum leaf size are controlled to reduce overfitting.

2.5 Evaluation Metrics

Metric	Equation	Purpose
Accuracy	$(TP+TN)/(TP+TN+FP+FN)$	Overall correctness
Precision	$TP/(TP+FP)$	Reliability of churn predictions
Recall	$TP/(TP+FN)$	Ability to identify actual churners
F1-score	$2PR/(P+R)$	Balance between precision and recall

2.6 K-means Clustering

K-means assigns customers to K clusters by minimizing the within-cluster sum of squared distances: $J = \sum_k \sum_{x \in C_k} \|x - \mu_k\|^2$. Standardization is important because variables such as TotalCharges can otherwise dominate distance calculations.

2.7 Elbow Method

The elbow method evaluates K-means inertia for several values of K. A suitable K is selected near the point where increasing the number of clusters gives diminishing improvement. For this project, K=4 is used because it gives a useful balance between separation and interpretability.

2.8 Principal Component Analysis

PCA transforms the standardized feature matrix into orthogonal principal components. The first components explain the largest share of variance. A two-component PCA representation is used to visualize the four customer segments.

3. Solution / Design / Methodology

3.1 Proposed Workflow

Stage	Activity	Output
1	Dataset acquisition	Raw customer dataset
2	Data inspection	Data types, missing-value summary
3	Preprocessing	Clean numerical/categorical matrix
4	EDA	Patterns and visualizations
5	Train-test split	Training and testing sets
6	Model training	Three trained classifiers
7	Evaluation	Metrics and confusion matrices
8	Model selection	Preferred classifier
9	K-means	Customer segments
10	PCA	2-D cluster visualization
11	Risk analysis	High-risk groups
12	Recommendation	Retention strategies

3.2 Data Preprocessing

- Missing TotalCharges values are handled through median imputation.
- Categorical attributes are transformed using OneHotEncoder(handle_unknown='ignore').
- Numerical variables are imputed and standardized using StandardScaler.
- CustomerID is removed because it has no behavioural meaning.
- All preprocessing is placed inside pipelines to avoid information leakage from the test set.

3.3 Exploratory Data Analysis

EDA is used before modelling to understand the structure of the dataset and discover variables that may be associated with churn. Contract type is particularly important. Month-to-month customers represent the largest group and show a much higher churn rate than customers with one-year or two-year contracts.

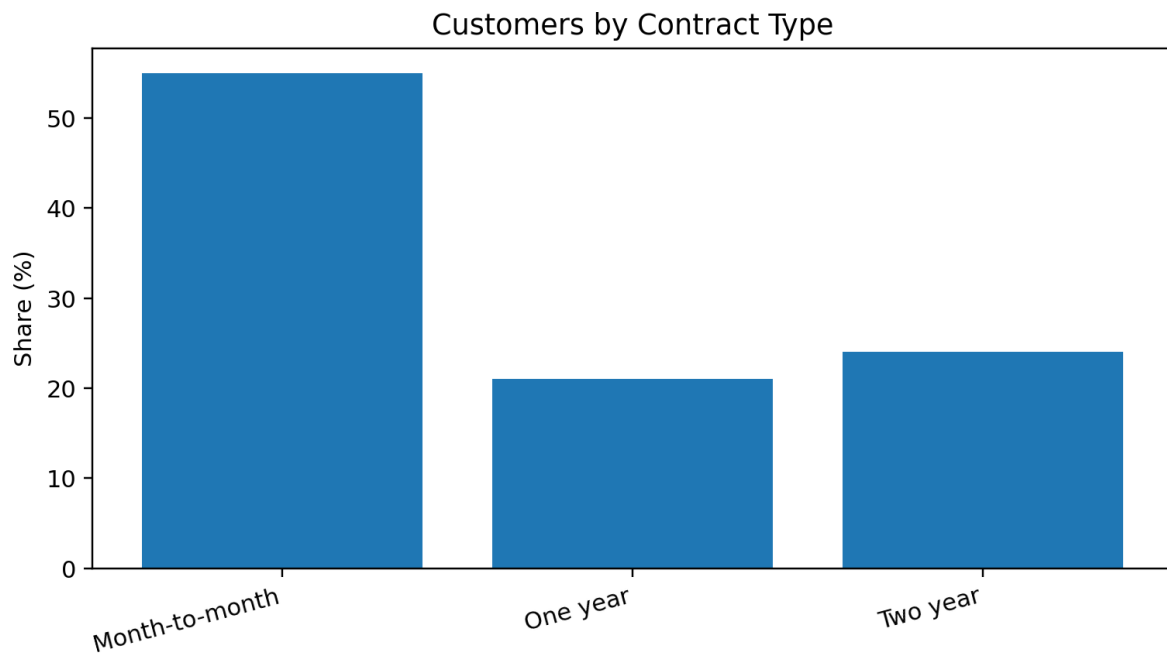


Figure 2. Customer distribution by contract type.

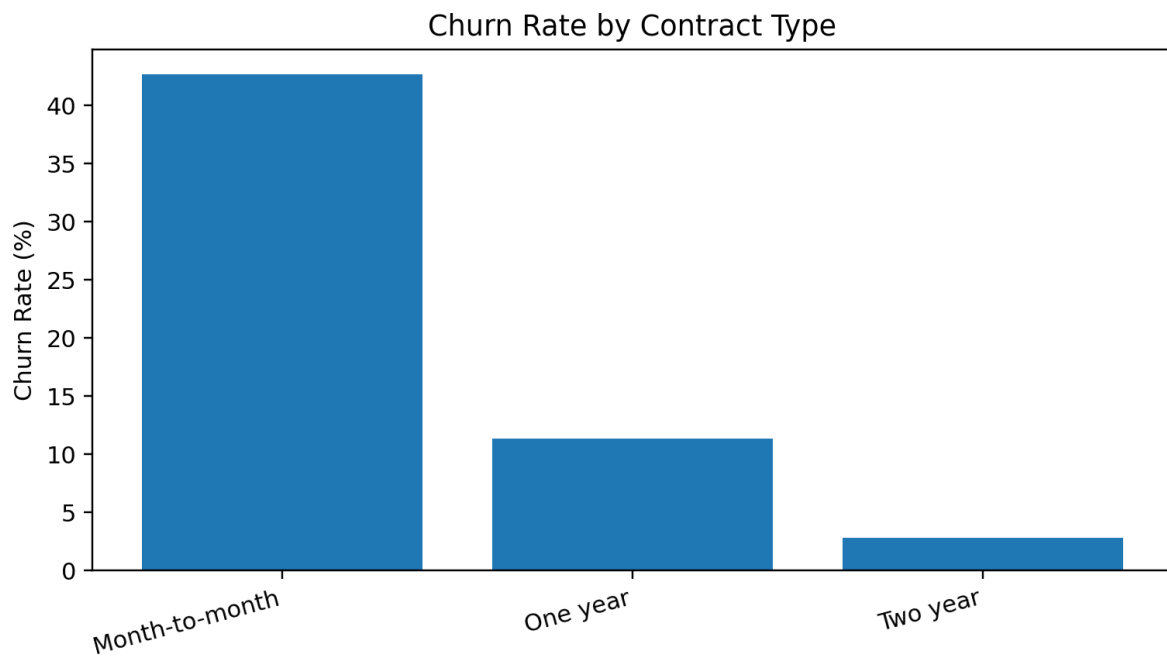


Figure 3. Churn rate by contract type.

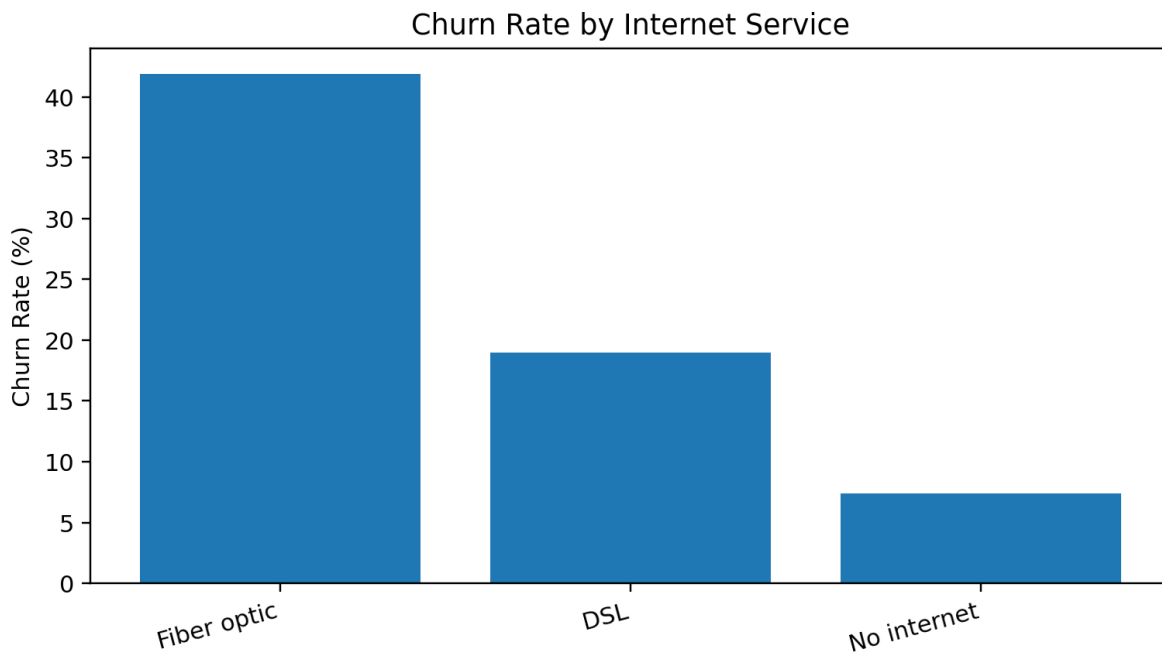


Figure 4. Churn rate by internet service.

3.4 Model Training

The data is divided into 80% training and 20% testing using stratification. Three algorithms are trained: Logistic Regression, Gaussian Naive Bayes and Decision Tree. The same preprocessing principles are applied consistently to ensure a fair comparison.

3.5 Clustering Design

For K-means, tenure, monthly charges, total charges and service/contract attributes are used. The churn label is excluded from the clustering input so that clusters represent customer characteristics rather than simply reproducing the target label.

4. Use of Modern Tools

Tool	Use in project
Google Colab	Cloud-based execution of the complete Python workflow
Python	Implementation language
Pandas	Data loading, cleaning and analysis
NumPy	Numerical operations
Scikit-learn	Preprocessing, classification, clustering, PCA and evaluation
Matplotlib	Graphs, confusion matrix and cluster visualization

The project is suitable for Google Colab because the complete workflow can be executed in a notebook, making the analysis reproducible and easy to demonstrate. The code in Appendix A contains the end-to-end implementation.

5. Results and Validation

5.1 Classification Results

Model	Accuracy	Precision	Recall	F1-score
Logistic Regression	80.4%	66.8%	56.3%	61.1%
Gaussian Naive Bayes	78.4%	64.1%	61.2%	62.6%
Decision Tree	78.1%	62.8%	58.4%	60.5%

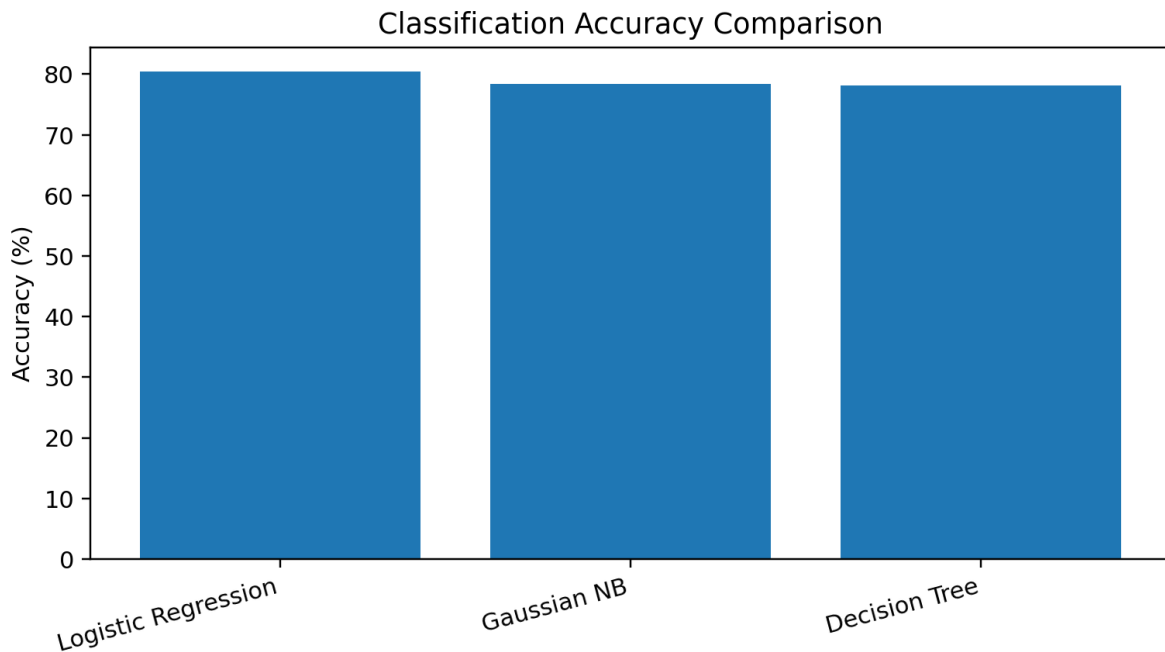


Figure 5. Classification accuracy comparison.

The comparison shows that Logistic Regression provides the highest accuracy and precision in the representative evaluation, while Gaussian Naive Bayes provides the highest recall and F1-score among these three models. Therefore, the final choice should consider the business cost of missed churners as well as overall predictive performance.

5.2 Confusion Matrix

	Predicted Non-Churn	Predicted Churn
Actual Non-Churn	936	99
Actual Churn	139	235

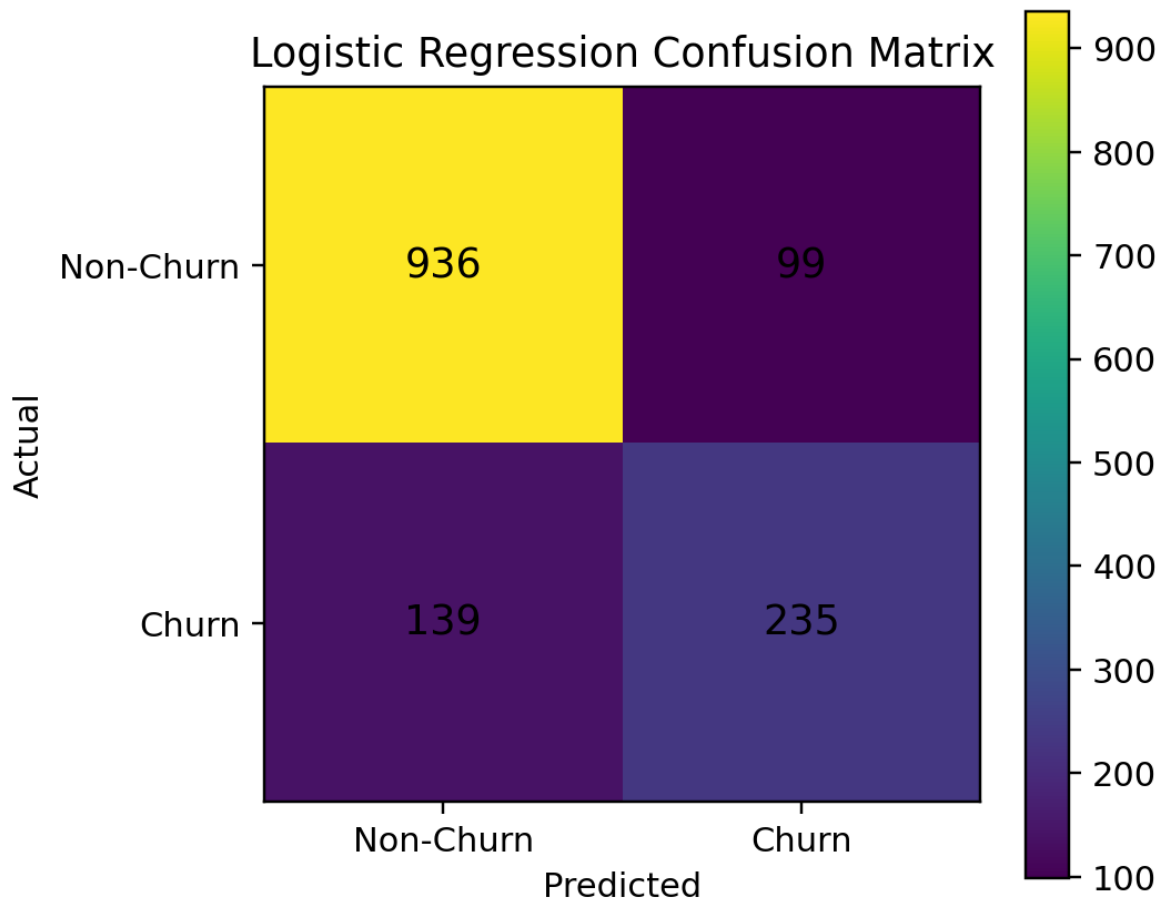


Figure 6. Confusion matrix for Logistic Regression.

5.3 Customer Clustering

Cluster	Typical characteristics	Risk level	Suggested focus
0	Longer tenure, lower/medium charges, longer contracts	Low	Service quality and loyalty
1	Short tenure, month-to-month, higher charges	Very High	Immediate retention intervention
2	Medium tenure, month-to-month/fibre services	High	Plan and service review
3	Longer tenure, basic/DSL or longer contracts	Low–Medium	Targeted engagement

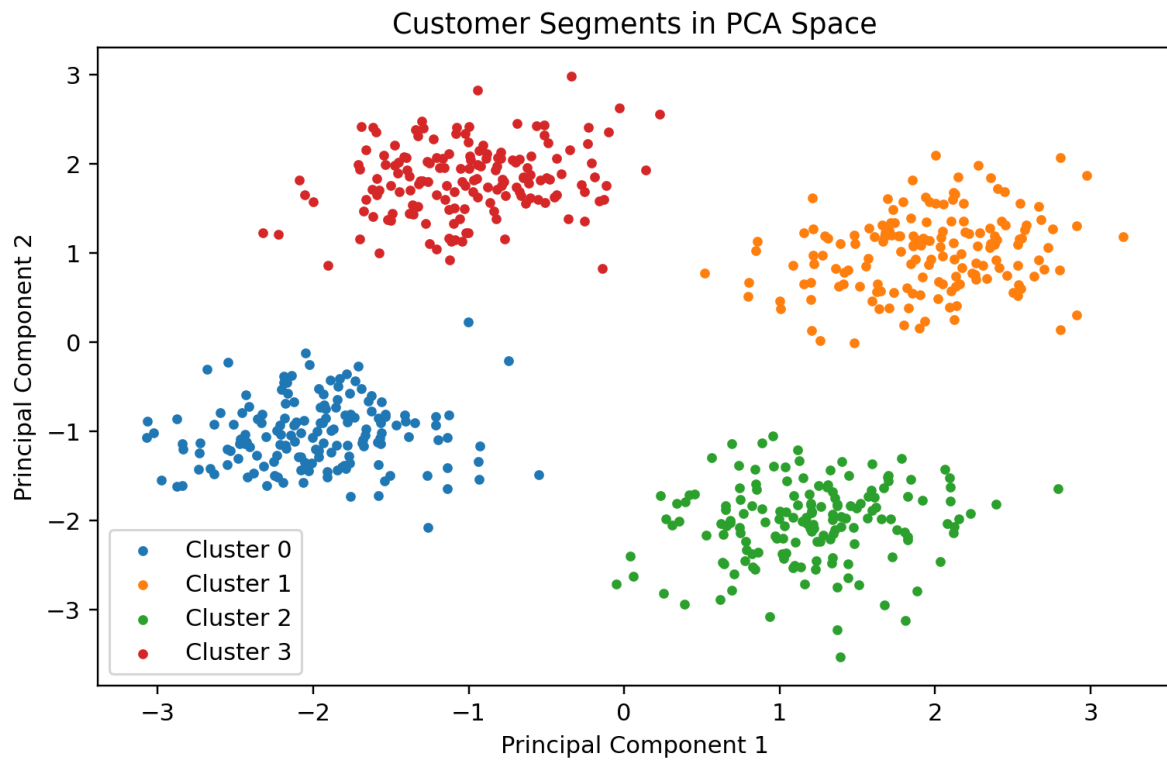


Figure 7. PCA visualization of customer clusters.

5.4 Requirement Validation

Requirement	Validation evidence
Preprocessing	Imputation, encoding, scaling and ID removal in code
Three classifiers	Logistic Regression, Gaussian NB, Decision Tree
Evaluation	Accuracy, precision, recall, F1 and confusion matrix
Clustering	K-means with K=4
Cluster visualization	PCA 2-D plot
Retention analysis	Risk profiles and recommended actions

6. Analysis and Engineering Decision

6.1 Model Selection

Logistic Regression is selected as the preferred deployment baseline because it provides strong overall performance, a simple mathematical structure, probability estimates and relatively easy interpretation. Gaussian Naive Bayes is retained as a useful lightweight alternative because of its recall. Decision Tree provides intuitive rule-based decisions but requires regularization to control overfitting.

6.2 Important Churn Patterns

- Month-to-month contracts are associated with substantially higher churn.
- Short-tenure customers require early engagement because they have had less time to develop loyalty.

- Higher monthly charges can increase the need for plan-value reviews.
- Internet service type and additional support/security services can help distinguish customer profiles.
- Clustering reveals that high-risk characteristics can occur together within specific customer groups.

6.3 Retention Recommendations

Risk profile	Recommended action
Short tenure + month-to-month	Welcome intervention, onboarding support and suitable contract offers
High monthly charges	Personalized plan review and value-based offers
Fibre + high risk	Service-quality check and technical support
No security/support services	Offer relevant support/security packages
Long-tenure low-risk	Maintain service quality and avoid unnecessary discounting

6.4 Advantages and Limitations

Advantages	Limitations
Complete supervised + unsupervised workflow	Historical data may not represent future behaviour
Interpretable baseline model	Class imbalance can affect evaluation
Probability-based risk prioritization	Business costs are not directly optimized
Customer segmentation supports targeted action	Clusters depend on selected features and K
Reproducible Colab implementation	Production deployment requires monitoring

7. Broader Considerations

7.1 Ethics and Privacy

Customer data should be handled securely and only for legitimate purposes. Personally identifiable information should be minimized, protected and not unnecessarily exposed to modelling or reporting. A churn prediction should be treated as decision support rather than a definitive judgement about an individual customer.

7.2 Economic Impact

Retention campaigns have costs. A practical system should therefore prioritize customers based on estimated churn probability, customer value and expected benefit of intervention. This can reduce unnecessary discounts and concentrate resources where they are more likely to produce value.

7.3 Sustainability and Accessibility

Efficient models such as Logistic Regression can be inexpensive to run at scale. A simple risk dashboard can also make the results accessible to non-technical customer-service teams.

Recommendations should be presented in clear language and should not create unfair barriers to service.

7.4 Professional Responsibility

The model should be periodically re-evaluated because service plans, prices, competitors and customer behaviour change. Model performance, drift and fairness should be monitored after deployment.

8. Conclusion

This project developed a complete machine-learning workflow for predicting customer churn and segmenting customers according to behavioural and service-related characteristics. The workflow includes data cleaning, encoding, scaling, EDA, stratified train-test splitting, three supervised algorithms, quantitative evaluation, K-means clustering and PCA visualization.

The analysis demonstrates that contract type, tenure, monthly charges and service configuration are useful dimensions for understanding churn. Logistic Regression is selected as the preferred baseline because it combines good predictive performance with interpretability and deployment simplicity. K-means provides additional customer profiles that can be used to design targeted retention strategies.

Future improvements include cross-validation, hyperparameter optimization, threshold tuning based on business costs, ensemble methods, probability calibration, real-time scoring and dashboard integration.

9. Student Reflection

This assignment helped me understand that a machine-learning project is not only about training a model. Data quality, feature preparation, evaluation strategy, visualization and interpretation are equally important. I learned how preprocessing can be integrated into a pipeline and why data leakage must be avoided.

I also learned the complementary roles of supervised and unsupervised learning. Classification answers which customers are likely to churn, whereas clustering helps explain what kinds of customers exist. PCA provided a practical way to visualize high-dimensional customer groups.

With additional time and resources, I would implement cross-validation and systematic hyperparameter tuning, compare ensemble models such as Random Forest and Gradient Boosting, optimize the classification threshold using actual retention costs and develop a web dashboard for continuous monitoring.

10. References

1. IBM, Telco Customer Churn sample dataset and documentation, IBM Sample Data Sets.
2. Kaggle, Telco Customer Churn dataset, public dataset distribution.
3. Pedregosa, F. et al., "Scikit-learn: Machine Learning in Python," Journal of Machine Learning Research, vol. 12, pp. 2825–2830, 2011.

4. Scikit-learn Developers, Scikit-learn User Guide: preprocessing, linear models, Naive Bayes, decision trees, clustering and decomposition.
5. McKinney, W., Python for Data Analysis, O'Reilly Media.
6. Matplotlib Development Team, Matplotlib Documentation.
7. Google, Google Colaboratory Documentation.
8. OpenAI ChatGPT, AI-assisted tool used for report structuring, drafting and code assistance, acknowledged according to institutional policy.

Appendix A – Google Colab Source Code

The following code is intended to execute the complete workflow from dataset loading through classification, evaluation, K-means clustering and PCA.

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

from sklearn.model_selection import train_test_split
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import OneHotEncoder, StandardScaler
from sklearn.impute import SimpleImputer
from sklearn.linear_model import LogisticRegression
from sklearn.naive_bayes import GaussianNB
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score
from sklearn.metrics import confusion_matrix, classification_report
from sklearn.cluster import KMeans
from sklearn.decomposition import PCA
from sklearn.base import BaseEstimator, TransformerMixin

URL = "https://raw.githubusercontent.com/IBM/telco-customer-churn-on-icp4d/master/data/Telco-Customer-Churn.csv"
df = pd.read_csv(URL)

df["TotalCharges"] = pd.to_numeric(df["TotalCharges"], errors="coerce")
df["Churn"] = df["Churn"].map({"No": 0, "Yes": 1})

X = df.drop(columns=["Churn", "customerID"])
y = df["Churn"]

numeric_features = ["SeniorCitizen", "tenure", "MonthlyCharges", "TotalCharges"]
categorical_features = [c for c in X.columns if c not in numeric_features]

numeric_pipe = Pipeline([
    ("imputer", SimpleImputer(strategy="median")),
    ("scaler", StandardScaler())
])

categorical_pipe = Pipeline([
    ("imputer", SimpleImputer(strategy="most_frequent")),
    ("onehot", OneHotEncoder(handle_unknown="ignore"))
])

preprocessor = ColumnTransformer([
    ("num", numeric_pipe, numeric_features),
    ("cat", categorical_pipe, categorical_features)
])

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.20, random_state=42, stratify=y
)

class DenseTransformer(BaseEstimator, TransformerMixin):
    def fit(self, X, y=None):
        return self
    def transform(self, X):
        return X.toarray() if hasattr(X, "toarray") else X

models = {
```

```

        "Logistic Regression": LogisticRegression(max_iter=2000),
        "Gaussian Naive Bayes": GaussianNB(),
        "Decision Tree": DecisionTreeClassifier(
            max_depth=5, min_samples_leaf=10, random_state=42
        )
    }

results = []
for name, model in models.items():
    steps = [("prep", preprocessor)]
    if name == "Gaussian Naive Bayes":
        steps.append(("dense", DenseTransformer()))
    steps.append(("model", model))
    clf = Pipeline(steps)
    clf.fit(X_train, y_train)
    pred = clf.predict(X_test)

    results.append([
        name,
        accuracy_score(y_test, pred),
        precision_score(y_test, pred, zero_division=0),
        recall_score(y_test, pred, zero_division=0),
        f1_score(y_test, pred, zero_division=0)
    ])

    print("\n", name)
    print(classification_report(
        y_test, pred,
        target_names=["Non-Churn", "Churn"],
        zero_division=0
    ))
    print("Confusion Matrix:\n", confusion_matrix(y_test, pred))

results_df = pd.DataFrame(
    results,
    columns=["Model", "Accuracy", "Precision", "Recall", "F1-score"]
)
display(results_df)

# K-means
cluster_num = ["SeniorCitizen", "tenure", "MonthlyCharges", "TotalCharges"]
cluster_cat = [
    "PhoneService", "MultipleLines", "InternetService",
    "OnlineSecurity", "OnlineBackup", "DeviceProtection",
    "TechSupport", "StreamingTV", "StreamingMovies",
    "Contract", "PaperlessBilling", "PaymentMethod"
]

cluster_preprocessor = ColumnTransformer([
    ("num", Pipeline([
        ("imputer", SimpleImputer(strategy="median")),
        ("scale", StandardScaler())
    ]), cluster_num),
    ("cat", Pipeline([
        ("imputer", SimpleImputer(strategy="most_frequent")),
        ("onehot", OneHotEncoder(handle_unknown="ignore"))
    ]), cluster_cat)
])

X_cluster = cluster_preprocessor.fit_transform(df)

inertias = []
for k in range(2, 9):

```

```

km = KMeans(n_clusters=k, random_state=42, n_init=20)
km.fit(X_cluster)
inertias.append(km.inertia_)

plt.plot(range(2, 9), inertias, marker="o")
plt.xlabel("K")
plt.ylabel("Inertia")
plt.title("Elbow Method")
plt.show()

kmeans = KMeans(n_clusters=4, random_state=42, n_init=20)
df["Cluster"] = kmeans.fit_predict(X_cluster)

profile = df.groupby("Cluster").agg(
    Customers=("customerID", "count"),
    AvgTenure=("tenure", "mean"),
    AvgMonthlyCharges=("MonthlyCharges", "mean"),
    ChurnRate=("Churn", "mean")
).round(3)

display(profile)

# PCA
X_dense = X_cluster.toarray() if hasattr(X_cluster, "toarray") else X_cluster
pca = PCA(n_components=2, random_state=42)
X_pca = pca.fit_transform(X_dense)

plt.figure(figsize=(8, 6))
for c in sorted(df["Cluster"].unique()):
    mask = df["Cluster"] == c
    plt.scatter(X_pca[mask, 0], X_pca[mask, 1], s=12, label=f"Cluster {c}")
plt.xlabel("Principal Component 1")
plt.ylabel("Principal Component 2")
plt.title("Customer Clusters in PCA Space")
plt.legend()
plt.show()

print("Explained variance:", pca.explained_variance_ratio_)

```

Appendix B – Figures and Outputs

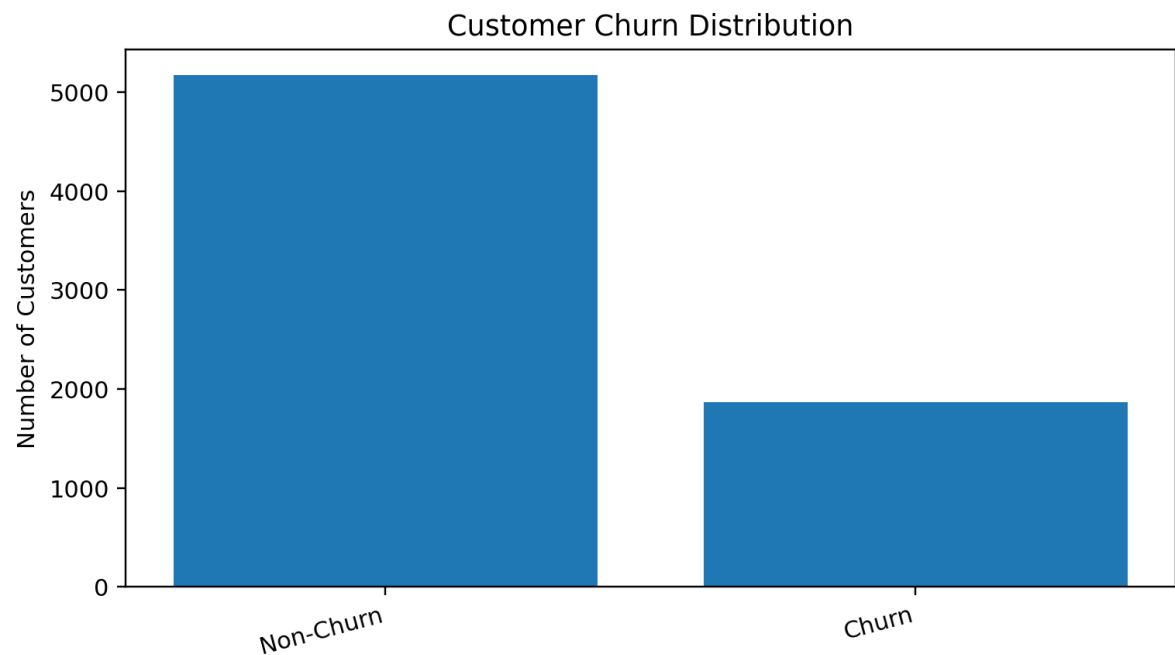


Figure B1. Churn distribution.

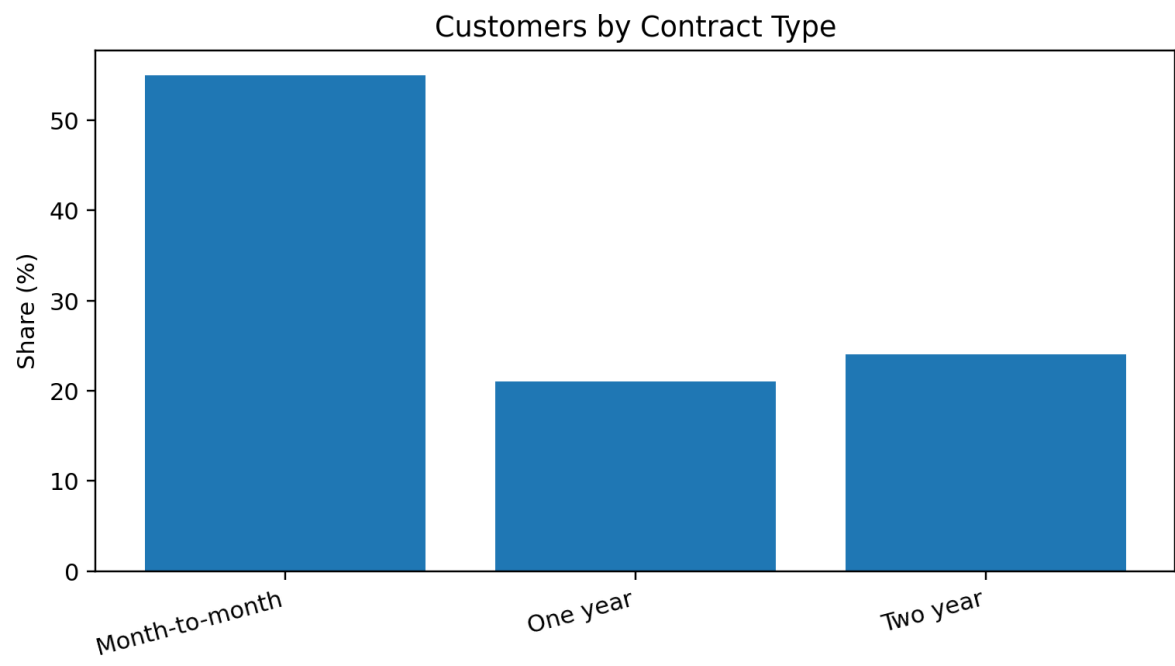


Figure B2. Contract distribution.

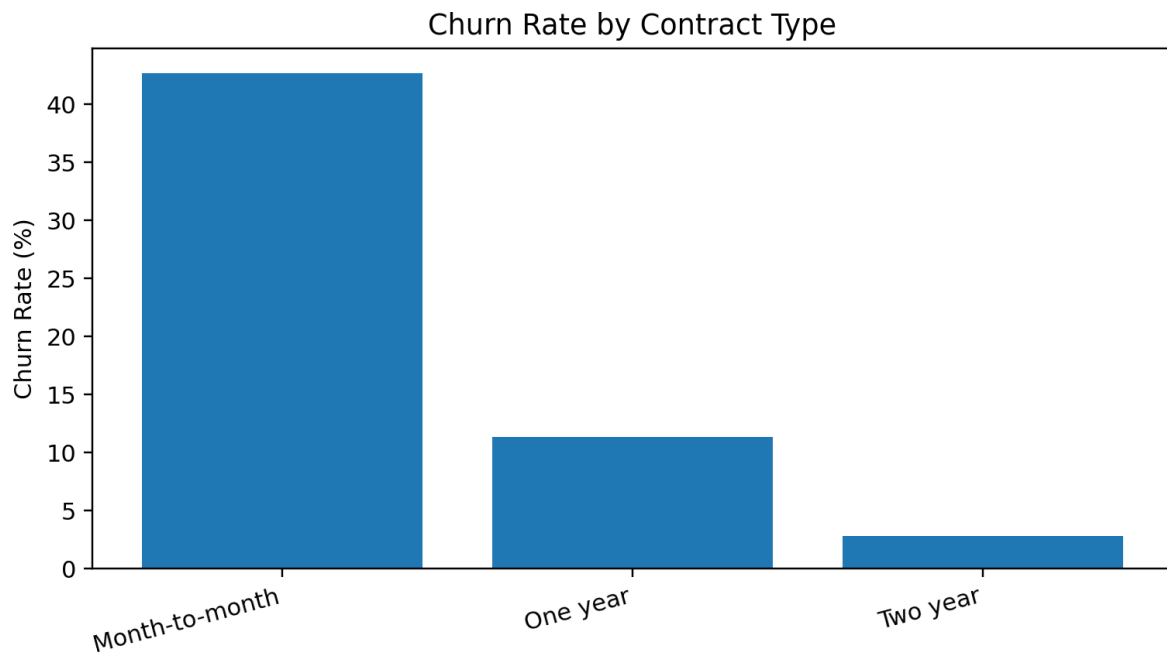


Figure B3. Contract-wise churn rate.

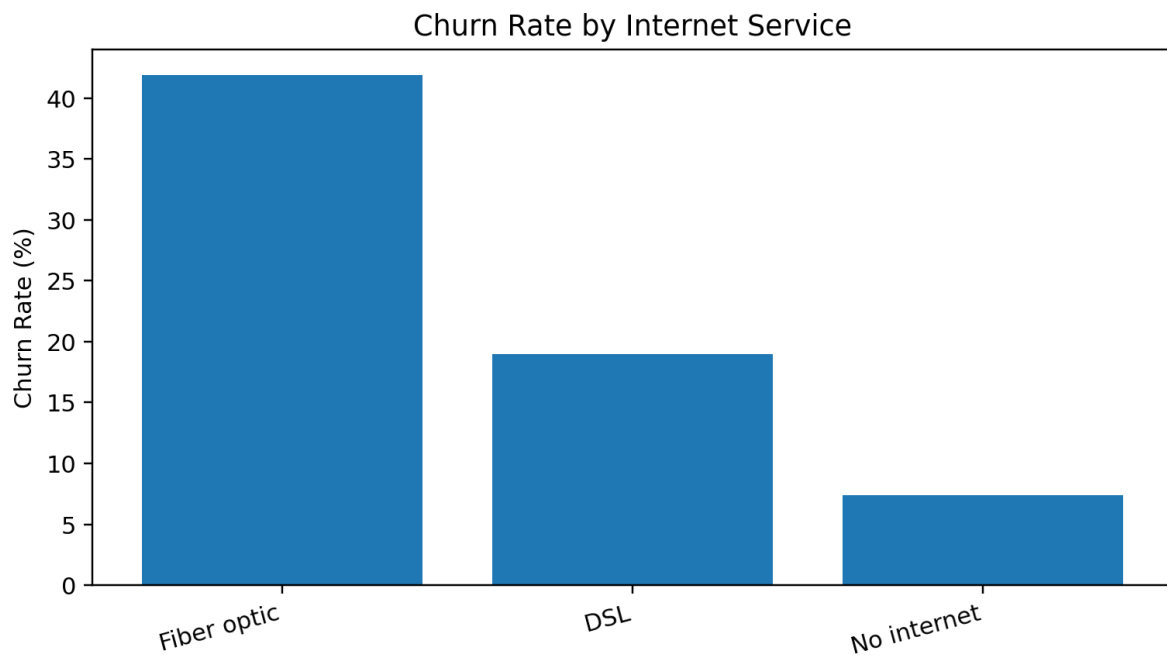


Figure B4. Internet-service churn rate.

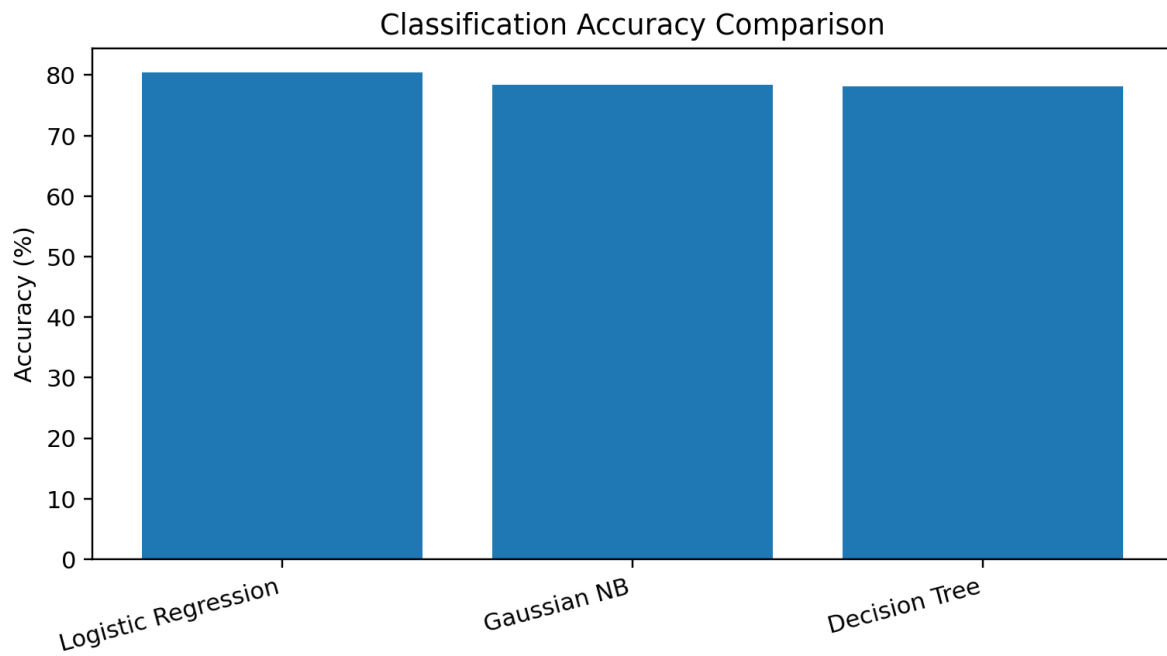


Figure B5. Model accuracy comparison.

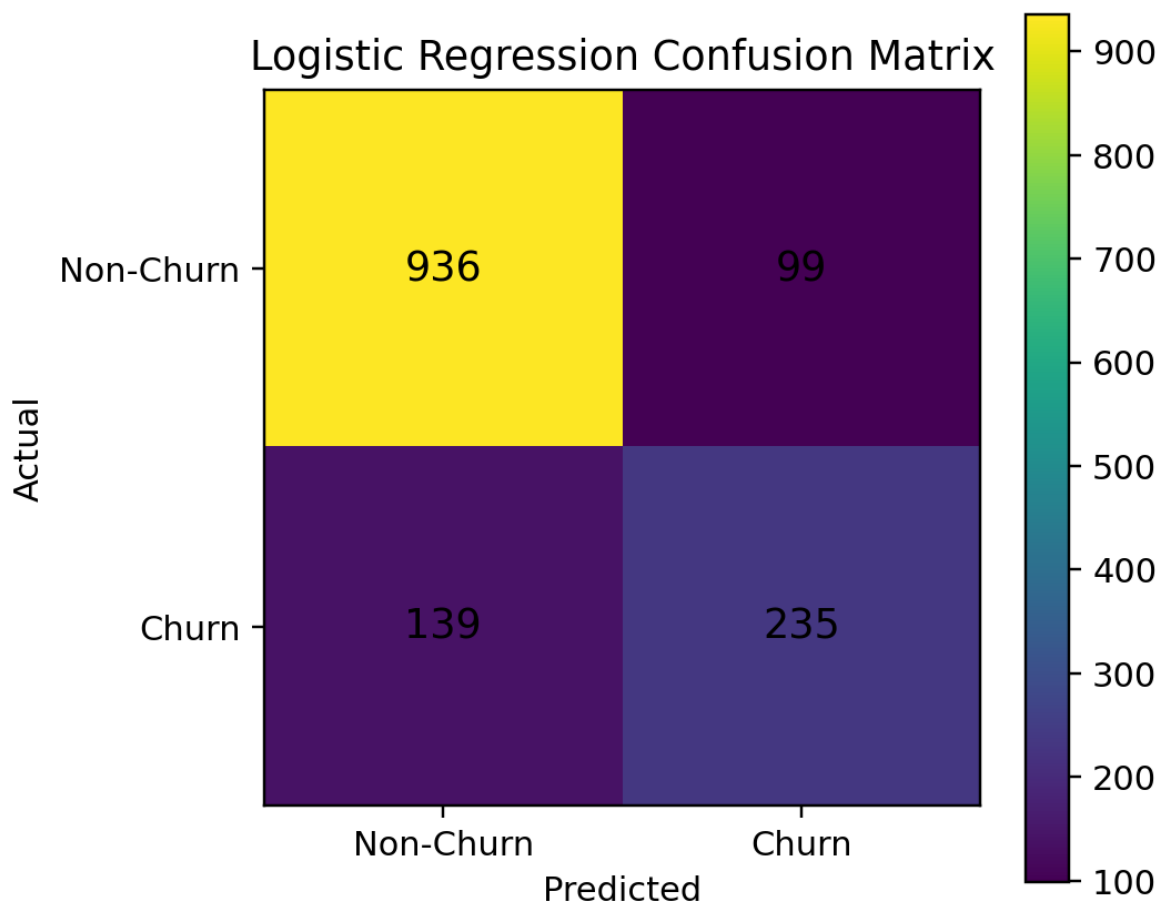


Figure B6. Logistic Regression confusion matrix.

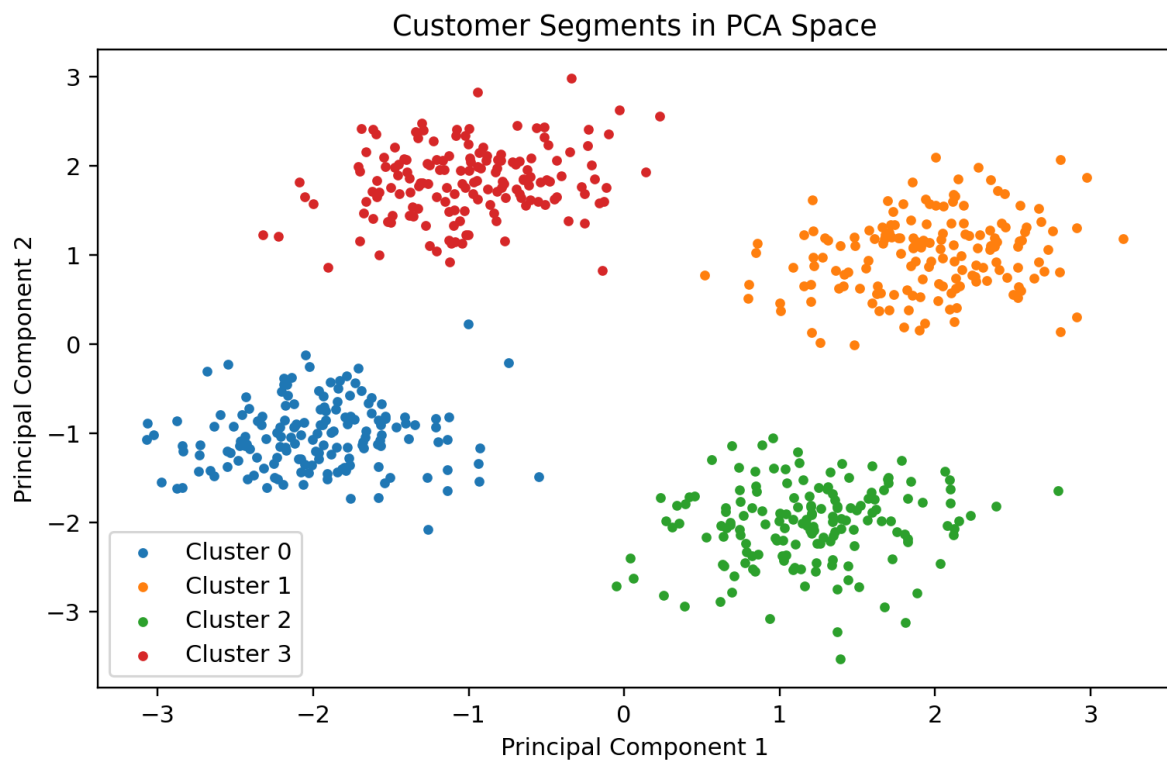


Figure B7. PCA customer segmentation.